

DSA with JavaScript

Beginner to Advanced • Interview Preparation Notes

How to use: Learn the definition, trace both examples by hand, implement without copying, then solve 5–15 problems per topic. Focus on patterns and complexity rather than memorizing answers.

Contents

1. DSA Foundations
2. Arrays & Strings
3. Searching, Sorting & Hashing
4. Linked Lists, Stacks & Queues
5. Trees, BST & Heap
6. Backtracking & Greedy
7. Graphs
8. Dynamic Programming
9. Advanced DSA
10. Interview Pattern Cheat Sheet

1. DSA Foundations

What is DSA?

Definition: Data Structures organize data; Algorithms are step-by-step procedures that process it. Interview DSA tests correctness, efficiency, edge cases, and your ability to explain trade-offs.

```
// Example 1: array traversal
const nums = [3, 7, 2];
for (const x of nums) console.log(x);

// Example 2: find maximum
function maxValue(a) {
  let best = a[0];
  for (const x of a) best = Math.max(best, x);
  return best;
}
```

Time & Space Complexity

Definition: Time complexity describes how running work grows with input size n . Space complexity describes how extra memory grows. Big-O focuses on growth, not exact milliseconds.

```
// O(1): direct access
const first = nums[0];

// O(n): one full traversal
function sum(a) {
  let s = 0;
  for (const x of a) s += x;
  return s;
}

// O(n^2): pair comparison
for (let i=0; i<nums.length; i++)
  for (let j=0; j<nums.length; j++) {}
```

Interview note: Ignore constants in Big-O. $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$ for large n .

Recursion

Definition: Recursion is when a function solves a problem by calling itself on a smaller version. Every recursive solution needs a base case and progress toward that base case.

```
// Example 1: factorial
function factorial(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
}

// Example 2: sum 1..n
function sumN(n) {
  if (n === 0) return 0;
  return n + sumN(n - 1);
}
```

2. Arrays & Strings

Arrays

Definition: An array stores an ordered sequence. JavaScript arrays provide indexed access. Common interview patterns include traversal, prefix sums, two pointers, and sliding windows.

```
// Example 1: reverse in place
function reverse(a) {
  let l=0, r=a.length-1;
  while(l<r) [a[l],a[r]]=a[r],a[l], l++, r--;
  return a;
}
```

```
// Example 2: prefix sums
function prefix(a) {
  const p=[0];
  for (const x of a) p.push(p[p.length-1]+x);
  return p;
}
```

Strings

Definition: Strings are sequences of characters. In JavaScript strings are immutable, so repeated concatenation or conversion to arrays may matter in some solutions.

```
// Example 1: palindrome
function isPalindrome(s) {
  let l=0,r=s.length-1;
  while(l<r) if(s[l++]!==s[r--]) return false;
  return true;
}

// Example 2: character frequency
function freq(s) {
  const m=new Map();
  for(const c of s) m.set(c,(m.get(c)||0)+1);
  return m;
}
```

Two Pointers & Sliding Window

Definition: Two pointers coordinate two indexes to avoid unnecessary nested loops. Sliding window maintains information about a contiguous range while moving through the input.

```
// Example 1: pair sum in sorted array
function hasPair(a,target){
  let l=0,r=a.length-1;
  while(l<r){
    const s=a[l]+a[r];
    if(s===target) return true;
    s<target ? l++ : r--;
  }
  return false;
}

// Example 2: max sum window of size k
function maxWindow(a,k){
  let s=0;
  for(let i=0;i<k;i++) s+=a[i];
  let best=s;
  for(let i=k;i<a.length;i++){
    s+=a[i]-a[i-k]; best=Math.max(best,s);
  }
  return best;
}
```

3. Searching, Sorting & Hashing

Binary Search

Definition: Binary search repeatedly removes half of a sorted search space. Typical complexity is $O(\log n)$. It also appears as 'binary search on answer'.

```
// Example 1: exact search
function binarySearch(a,t){
  let l=0,r=a.length-1;
  while(l<=r){
    const m=l+Math.floor((r-l)/2);
    if(a[m]===t) return m;
    if(a[m]<t) l=m+1; else r=m-1;
  }
  return -1;
}
```

```
// Example 2: first >= target (lower bound)
function lowerBound(a,t){
  let l=0,r=a.length;
  while(l<r){
    const m=Math.floor((l+r)/2);
    if(a[m]<t) l=m+1; else r=m;
  }
  return l;
}
```

Sorting

Definition: Sorting rearranges values by an order. Learn elementary sorts for concepts, but master merge sort, quicksort ideas, and JavaScript's comparator-based `Array.sort` for interviews.

```
// Example 1: bubble sort
function bubble(a){
  a=[...a];
  for(let i=0;i<a.length;i++){
    for(let j=0;j<a.length-1-i;j++){
      if(a[j]>a[j+1]) [a[j],a[j+1]]=a[j+1],a[j];
    }
  }
  return a;
}
```

```
// Example 2: numeric built-in sort
const sorted = [10,2,30,4].sort((a,b)=>a-b);
```

Hashing

Definition: Hash maps and sets provide average $O(1)$ lookup/insert and are central to frequency, duplicate, complement, grouping, and prefix-sum problems.

```
// Example 1: Two Sum
function twoSum(a,target){
  const m=new Map();
  for(let i=0;i<a.length;i++){
    const need=target-a[i];
    if(m.has(need)) return [m.get(need),i];
    m.set(a[i],i);
  }
}
```

```
// Example 2: duplicate
function hasDuplicate(a){
  return new Set(a).size !== a.length;
}
```

4. Linked Lists, Stacks & Queues

Linked List

Definition: A linked list stores nodes connected by references. Unlike arrays, nodes need not occupy contiguous memory. Core patterns: dummy node, reversal, fast/slow pointers.

```
class ListNode {
  constructor(val,next=null){ this.val=val; this.next=next; }
}
// Example 1: reverse
function reverseList(head){
  let prev=null,cur=head;
  while(cur){ const next=cur.next; cur.next=prev; prev=cur; cur=next; }
  return prev;
}
// Example 2: cycle detection
function hasCycle(head){
  let slow=head,fast=head;
  while(fast&&fast.next){
    slow=slow.next; fast=fast.next.next;
    if(slow===fast) return true;
  }
  return false;
}
```

```
}
```

Stack

Definition: A stack is LIFO: last in, first out. JavaScript arrays can implement stacks with push/pop. Common uses include parentheses, monotonic stacks, DFS, and expression processing.

```
// Example 1
const stack=[];
stack.push(10); stack.push(20);
stack.pop(); // 20

// Example 2: valid parentheses
function valid(s){
  const st=[], pair={'(':')','[':']','{':'}';
  for(const c of s){
    if(['(', '[', '{'].includes(c)) st.push(c);
    else if(st.pop()!==pair[c]) return false;
  }
  return st.length===0;
}
```

Queue

Definition: A queue is FIFO: first in, first out. For performance, avoid repeated Array.shift() on large inputs; use an index pointer or implement a queue.

```
// Example 1: efficient array-backed queue
const q=[], head={i:0};
q.push(10); q.push(20);
const x=q[head.i++];

// Example 2: BFS skeleton
function bfs(start,graph){
  const q=[start], seen=new Set([start]);
  for(let i=0;i<q.length;i++){
    const u=q[i];
    for(const v of graph[u]||[])
      if(!seen.has(v)){seen.add(v);q.push(v);}
  }
}
```

5. Trees, BST & Heap

Binary Trees

Definition: A binary tree node has at most two children. DFS uses preorder/inorder/postorder; BFS visits level by level. Recursion is a natural tree tool.

```
// Example 1: max depth
function depth(root){
  if(!root) return 0;
  return 1+Math.max(depth(root.left),depth(root.right));
}

// Example 2: inorder
function inorder(root,out=[]){
  if(!root) return out;
  inorder(root.left,out); out.push(root.val); inorder(root.right,out);
  return out;
}
```

Binary Search Tree

Definition: A BST maintains ordering: values in the left subtree are smaller and values in the right subtree are larger under the usual distinct-key convention.

```
// Example 1: search
function searchBST(root,x){
  while(root){
```

```

    if(root.val===x) return root;
    root=x<root.val?root.left:root.right;
  }
  return null;
}

// Example 2: validate with bounds
function validBST(n, lo=-Infinity, hi=Infinity){
  if(!n) return true;
  if(n.val<=lo||n.val>=hi) return false;
  return validBST(n.left, lo, n.val)&&validBST(n.right, n.val, hi);
}

```

Heap / Priority Queue

Definition: A heap efficiently retrieves the minimum or maximum priority element. Core operations are typically $O(\log n)$; peek is $O(1)$. JavaScript has no universal built-in priority queue, so interviews may require an implementation.

```

// Example 1 concept:
// min-heap root = smallest element
// parent(i) = Math.floor((i-1)/2)
// left(i)=2*i+1, right(i)=2*i+2

// Example 2 use cases:
// kth largest -> keep min-heap of size k
// Dijkstra -> min-priority queue by distance

```

6. Backtracking & Greedy

Backtracking

Definition: Backtracking explores a decision tree, chooses an option, recurses, and undoes the choice. It is common for combinations, permutations, N-Queens, and constraint problems.

```

// Example 1: subsets
function subsets(a){
  const ans=[],path=[];
  function dfs(i){
    if(i===a.length){ans.push([...path]);return;}
    dfs(i+1);
    path.push(a[i]); dfs(i+1); path.pop();
  }
  dfs(0); return ans;
}

// Example 2: permutations pattern
// choose unused -> recurse -> unchoose

```

Greedy

Definition: A greedy algorithm makes the locally best choice at each step. It is valid only when the problem has a property proving those local choices lead to a global optimum.

```

// Example 1: interval scheduling idea
// sort by end time; repeatedly choose next compatible interval

// Example 2: stock unlimited transactions
function profit(a){
  let p=0;
  for(let i=1;i<a.length;i++){
    if(a[i]>a[i-1]) p+=a[i]-a[i-1];
  }
  return p;
}

```

7. Graphs

Graph Fundamentals

Definition: A graph contains vertices and edges. It may be directed/undirected, weighted/unweighted, cyclic/acyclic. Adjacency lists are the common interview representation.

```
// Example 1: adjacency list
const graph = {
  A:['B','C'], B:['A','D'], C:['A'], D:['B']
};

// Example 2: DFS
function dfs(u,g,seen=new Set()){
  if(seen.has(u)) return;
  seen.add(u);
  for(const v of g[u]||[]) dfs(v,g,seen);
  return seen;
}
```

Topological Sort & Shortest Paths

Definition: Topological sorting orders a DAG so every directed edge points forward. Dijkstra finds shortest paths with non-negative edge weights; Bellman-Ford can handle negative edges.

```
// Example 1: Kahn's algorithm idea
// indegree[] -> enqueue zero-indegree nodes
// remove node -> decrement neighbors -> enqueue new zeros

// Example 2: Dijkstra idea
// dist[start]=0
// repeatedly extract smallest-distance node
// relax: if dist[u]+w < dist[v], update dist[v]
```

Union Find & MST

Definition: Disjoint Set Union tracks connected components using parent links, path compression, and union by rank/size. Kruskal uses DSU to build a minimum spanning tree.

```
// Example 1: find with compression
function find(x,parent){
  if(parent[x]!==x) parent[x]=find(parent[x],parent);
  return parent[x];
}

// Example 2: Kruskal idea
// sort edges by weight
// add edge if endpoints are in different DSU components
```

8. Dynamic Programming

DP Fundamentals

Definition: Dynamic programming applies when subproblems overlap and optimal solutions can be built from smaller solutions. Learn recursion -> memoization -> tabulation -> space optimization.

```
// Example 1: Fibonacci tabulation
function fib(n){
  if(n<2) return n;
  let a=0,b=1;
  for(let i=2;i<=n;i++) [a,b]=[b,a+b];
  return b;
}

// Example 2: climbing stairs
function climb(n){
  let a=1,b=1;
  for(let i=2;i<=n;i++) [a,b]=[b,a+b];
  return b;
}
```

Knapsack / Subsequence DP

Definition: Knapsack-style DP decides whether/how to take items under constraints. Subsequence DP includes LCS, LIS, subset sum, coin change, and related state-transition problems.

```
// Example 1: 0/1 knapsack state
// dp[i][cap] = best using first i items with capacity cap
// skip item OR take it once if it fits

// Example 2: LCS state
// if a[i-1]==b[j-1]: 1+dp[i-1][j-1]
// else: max(dp[i-1][j], dp[i][j-1])
```

9. Advanced DSA

Trie

Definition: A trie stores strings character-by-character, sharing common prefixes. Useful for prefix search, dictionaries, autocomplete, and some word-search problems.

```
// Example 1: trie node
class TrieNode {
  constructor(){ this.next=new Map(); this.end=false; }
}
// Example 2:
// insert: walk/create each character
// search: walk each character and check end flag
```

Bit Manipulation

Definition: Bits can represent flags and compact states. Important operators in JS: &: AND, |: OR, ^: XOR, ~: NOT, << and >> shifts. JS bitwise operators operate on 32-bit integers.

```
// Example 1: odd/even
const isOdd = n => (n & 1) === 1;

// Example 2: single number when others occur twice
function single(a){
  let x=0;
  for(const n of a) x ^= n;
  return x;
}
```

Segment Tree / Fenwick Tree

Definition: These structures answer repeated range queries with updates faster than recomputing every range. Segment trees are flexible; Fenwick trees are compact for prefix-based operations.

```
// Example 1: segment tree typical:
// build O(n), query O(log n), point update O(log n)

// Example 2: Fenwick update/query idea:
// update: i += i & -i
// prefix query: i -= i & -i
```

String Algorithms

Definition: Advanced string matching avoids repeatedly comparing every character. Know the purpose of KMP, Z algorithm, and rolling-hash/Rabin-Karp techniques.

```
// Example 1: KMP
// build LPS/prefix table, then reuse matched-prefix information

// Example 2: Rabin-Karp
// compare rolling hashes, verify characters on hash match
```

10. Interview Pattern Cheat Sheet

Recognize the Pattern

Definition: Use constraints and wording to identify likely techniques. Always discuss brute force first when useful, then improve it and state complexity.

```
// Common clues:
// sorted array      -> binary search / two pointers
// contiguous subarray -> sliding window / prefix sum
// frequency / lookup -> Map / Set
// top K             -> heap
// hierarchy          -> tree DFS/BFS
// shortest unweighted -> BFS
// dependencies       -> topological sort
// all combinations   -> backtracking
// repeated subproblems -> DP
```

JavaScript Interview Essentials

Definition: Be fluent with Array, Map, Set, strings, destructuring, sorting comparators, loops, and recursion. Know language-specific performance traps.

```
const m = new Map();
m.set('a', 1);
m.get('a');
m.has('a');

const s = new Set([1,2,2]); // {1,2}
const a = [3,1,2].sort((x,y)=>x-y);

// Prefer index-based queue traversal over repeated shift()
// when performance matters.
```

Recommended Study Order

1. JavaScript fundamentals + Big-O
2. Arrays + Strings + Hashing
3. Two Pointers + Sliding Window
4. Searching + Sorting
5. Recursion + Backtracking
6. Linked Lists + Stack + Queue
7. Trees + BST + Heap
8. Greedy
9. Graphs
10. Dynamic Programming
11. Trie + Bit Manipulation + Segment/Fenwick Trees + String Algorithms

Interview routine: For every problem: clarify input/edge cases → explain brute force → derive optimized approach → code → dry run → state time and space complexity.